HOWARD Help Parameters

Contents

1 Introduction

HOWARD Parameters JSON file defines parameters to process annotations, calculations, prioritizations, conversions and queries.

Examples:

Parameters for annotation, calculation, prioritization, HGVS annotation and export

```
{
   "annotation": {
      "parquet": {
         "annotations": {
            "/path/to/database3.parquet": {
               "annotation_fields": {
                  "field1": null,
                  "field2": "field2_renamed"
               }
            },
            "/path/to/database4.vcf.gz": {
               "annotation_fields": {
                  "INFO": null
               }
            },
            "/path/to/database5.bed.gz": {
               "annotation_fields": {
                  "INFO": null
               }
            }
         }
```

```
},
"bcftools": {
   "annotations": {
      "/path/to/database6.vcf.gz": {
         "annotation_fields": {
            "field1": null,
            "field2": "field2_renamed"
         }
      },
      "/path/to/database7.bed": {
         "annotation_fields": {
            "INFO": null
         }
      }
   }
},
"annovar": {
   "annotations": {
      "annovar_annotation1": {
         "annotation_fields": {
            "field1": null,
            "field2": "field2_renamed"
         },
         "options": {
            "protocol": "annovar_keyword1",
            "operation": "gx",
            "genebase": "-splicing_threshold 3 -indel_splicing_threshold 3 -hgvs",
            "arguments": "",
            "xref": "/path/to/xref/file",
            "options": ""
         }
      },
      "annovar_annotation2": {
         "annotation_fields": {
            "INFO": null,
            "field2": "field2_renamed"
         },
         "options": {
            "protocol": "annovar_keyword2",
            "operation": "f",
            "genebase": "",
            "arguments": "",
            "xref": "",
            "options": ""
         }
      }
   }
},
"snpeff": {
   "options": " -hgvs -spliceSiteSize 3 -lof -oicr "
},
"exomiser": {
   "release": "2109",
   "transcript_source": "refseq",
   "hpo": ["HP:0001156", "HP:0001363", "HP:0011304", "HP:0010055"]
},
"hgvs": {
   "full_format": true,
   "use_exon": true
```

```
        }
        "options": {
           "append": true
        }
     },
     "calculation": {
        "operation1": null,
        "operation2": {
          "options": {
            "option1": "value1",
            "option2": "value2"
          }
        }
     },
     "prioritization": {
        "prioritizations": "config/prioritization_profiles.json",
        "profiles": ["GENOME", "GERMLINE"],
        "default_profile": "GERMLINE",
        "pzfields": ["PZScore", "PZFlag", "PZComment"],
        "prioritization_score_mode": "VaRank"
     },
     "export": {
        "include_header": true
     }
  }
```

2 pipeline

Pipeline options to define the order of operations to process as a list of steps.

Two pipeline declaration modes are available:

- Inline mode (recommended): each step contains one or multiple named operations, each embedding its own configuration, a '_tool' key set to one of: 'annotation', 'calculation', 'prioritization', and an optional '_description' key to document the operation.
- Referenced mode (alternative, legacy-compatible): each step maps a section name to a tool type (e.g. {"annotation_frequency": "annotation"}), and parameters are defined in top-level sections of the parameter JSON file. An optional '_description' key can also be added in each referenced section. The '_tool' key can be used instead of define it on pipeline section (e.g. {"annotation_frequency": None} with section "annotation_frequency" including '_tool' key as "annotation").

The referenced mode is useful to reuse the same operation configuration in multiple pipeline steps, to define a pipeline with a minimal or choosen steps.

By default, if no pipeline is provided, operations are processed in referenced mode in the specific order: "annotation", "calculation", "prioritization."

For more information about steps and operations, see each tool's documentation below.

Examples:

Recommended: pipeline in inline mode with full step descriptions

```
{
   "pipeline": {
     "steps": [
       {
         "annotation_dbsnp": {
           "_tool": "annotation",
           "_description": "Annotate variants with dbSNP annotation.",
           "parquet": {
             "annotations": {
```

```
            "tests/databases/annotations/current/hg19/avsnp150.parquet": {"INFO": null}
          }
        }
      },
      "annotation_dbNSFP": {
        "_tool": "annotation",
        "_description": "Annotate variants with dbNSFP and COSMIC databases.",
        "parquet": {
          "annotations": {
            "tests/databases/annotations/current/hg19/dbnsfp42a.parquet": {"INFO": null}
          }
        },
        "bcftools": {
          "annotations": {
            "tests/databases/annotations/current/hg19/cosmic70.vcf.gz": {"INFO": null}
          }
        }
      }
    },
    {
      "calculation_on_variants": {
        "_tool": "calculation",
        "_description": "Compute variant and genotype metrics such as vartype and VAF.",
        "calculations": {"vartype": null, "VAF": ""},
        "calculation_config": "config/calculations_config.json"
      }
    },
    {
      "prioritization": {
        "_tool": "prioritization",
        "_description": "Prioritize variants using configured profiles and scoring mode.",
        "profiles": ["default", "GERMLINE"],
        "prioritization_config": "config/prioritization_profiles.json",
        "pzfields": ["PZScore", "PZFlag", "PZComment"],
        "prioritization_score_mode": "VaRank"
      }
    }
  ]
 },
}
```

Alternative: pipeline in referenced mode ("prioritization_step" tool is defined in the "prioritization_step" referenced top-level section)

```
{
 "pipeline": {
  "steps": [
    {"annotation_step": "annotation"},
    {"calculation_step": "calculation"},
    {"prioritization_step": null}
  ]
 },
}
```

Corresponding operation sections for referenced mode

```
{
 "annotation_step": {
  "_description": "Annotate variants with configured annotation databases.",
  "parquet": {
    "annotations": {
```

```
            "/path/to/database.parquet": {
              "annotation_fields": {
                "INFO": null
              }
            }
          }
        }
      },
      "calculation_step": {
        "_description": "Perform configured calculation operations on variants and genotypes.",
        "calculations": {
          "operation1": null,
          "operation2": {
            "options": {
              "option1": "value1",
              "option2": "value2"
            }
          }
        },
        "calculation_config": "calculation_config.json"
      },
      "prioritization_step": {
        "_tool": "prioritization",
        "_description": "Prioritize variants with configured profiles and scoring mode.",
        "prioritizations": "config/prioritization_profiles.json",
        "profiles": ["GENOME", "GERMLINE"],
        "default_profile": "GERMLINE",
        "pzfields": ["PZScore", "PZFlag", "PZComment"],
        "prioritization_score_mode": "VaRank"
      }
    }
```

2.1 steps

List of steps to process in the pipeline.

Each step is a dict and can use one of two formats:

- Inline format (recommended): operation name as key mapped to an operation object containing '_tool', optional '_description', and operation parameters.
- Referenced format (alternative): section name as key (e.g. "annotation", "calculation", "prioritization", "annotation_core", "annotation_frequency") mapped to the tool type ("annotation", "calculation", "prioritization"); the corresponding referenced section may include an alternative '_tool' key, and an optional '_description' key.

Only tool types "annotation", "calculation" and "prioritization" are available.

By default, all steps are processed in the order: "annotation", "calculation", "prioritization".

Beware that some referenced steps can be skipped if no corresponding section is provided in the parameter JSON file (e.g. '{"annotation_frequency": "annotation"}' needs "annotation_frequency" section in parameter JSON file).

Type: `list`

Default: `[ {"annotation": "annotation"}, {"calculation": "calculation"}, {"prioritization": "prioritization"} ]`

Examples:

Recommended: inline full descriptions

```
{
  "steps": [
    {
```

```
      "annotation_dbsnp": {
        "_tool": "annotation",
        "_description": "Annotate variants with dbSNP annotation.",
        "parquet": {
          "annotations": {
            "tests/databases/annotations/current/hg19/avsnp150.parquet": {"INFO": null}
          }
        }
      }
    },
    {
      "calculation_on_variants": {
        "_tool": "calculation",
        "_description": "Compute variant and genotype metrics such as vartype and VAF.",
        "calculations": {"vartype": null, "VAF": ""},
        "calculation_config": "config/calculations_config.json"
      }
    },
    {
      "prioritization": {
        "_tool": "prioritization",
        "_description": "Prioritize variants using configured profiles and scoring mode.",
        "profiles": ["default", "GERMLINE"]
      }
    }
  ]
}
```

Alternative: referenced format (default order) with tool types

```
{
  "steps": [
    {"annotation": "annotation"},
    {"calculation": "calculation"},
    {"prioritization": "prioritization"}
  ]
}
```

Referenced sections in parameter JSON file (in main section level)

```
{
  "annotation": {"_tool": "annotation", "_description": "Annotate variants with configured annotation d
  "calculation": {"_tool": "calculation", "_description": "Perform configured calculation operations on
  "prioritization": {"_tool": "prioritization", "_description": "Prioritize variants with configured pr
  "annotation_core": {"_tool": "annotation", "_description": "Core annotation step", ...},
  "annotation_frequency": {"_tool": "annotation", "_description": "Frequency annotation step", ...},
  "annotation_scores": {"_tool": "annotation", "_description": "Scores annotation step", ...},
  "calculation": {"_tool": "calculation", "_description": "Main calculation step", ...},
  "calculation_frequency": {"_tool": "calculation", "_description": "Frequency calculation step", ...},
  "calculation_final": {"_tool": "calculation", "_description": "Final calculation step", ...},
  "prioritization": {"_tool": "prioritization", "_description": "Prioritization step", ...}
}
```

Alternative: referenced format with multiple operation sections, with tool types and optional '_description' in each section

```
{
  "steps": [
    {"annotation_core": null},
    {"calculation": null},
    {"annotation_frequency": null},
    {"annotation_scores": null},
```

```
      {"calculation_frequency": null},
      {"prioritization": null},
      {"calculation_final": null}
    ]
}
```

3 annotation

Annotation process using HOWARD algorithms or external tools.

For HOWARD Parquet algorithm, provide the list of database files available (formats such as Parquet, VCF, TSV, duckDB, JSON) and select fields (rename possible, 'INFO' keyword for all fields), or use 'ALL' keyword to detect available databases.

For external tools, such as Annovar, snpEff, Exomiser and Docker-based annotation tools, specify parameters such as annotation keywords (Annovar), options (depending on the tool), and selected fields (BCFtools and Annovar, field rename available).

Examples:

Annotation with multiple tools in multiple formats with multiple options

```
{
   "annotation": {
      "parquet": {
         "annotations": {
            "/path/to/database3.parquet": {
               "annotation_fields": {
                  "field1": null,
                  "field2": "field2_renamed"
               }
            },
            "/path/to/database4.vcf.gz": {
               "annotation_fields": {
                  "INFO": null
               }
            },
            "/path/to/database5.bed.gz": {
               "annotation_fields": {
                  "INFO": null
               }
            }
         }
      }
      "bcftools": {
         "annotations": {
            "/path/to/database6.vcf.gz": {
               "annotation_fields": {
                  "field1": null,
                  "field2": "field2_renamed"
               }
            },
            "/path/to/database7.bed": {
               "annotation_fields": {
                  "INFO": null
               }
            }
         }
      }
      "annovar": {
         "annotations": {
            "annovar_annotation1": {
```

```
                "annotation_fields": {
                    "field1": null,
                    "field2": "field2_renamed"
                },
                "options": {
                    "protocol": "annovar_keyword1",
                    "operation": "gx",
                    "genebase": "-splicing_threshold 3 -indel_splicing_threshold 3 -hgvs",
                    "arguments": "",
                    "xref": "/path/to/xref/file",
                    "options": ""
                }
            },
            "annovar_annotation2": {
                "annotation_fields": {
                    "INFO": null,
                    "field2": "field2_renamed"
                }
                "options": {
                    "protocol": "annovar_keyword2",
                    "operation": "f",
                    "genebase": "",
                    "arguments": "",
                    "xref": "",
                    "options": ""
                }
            }
        }
        "snpeff": {
            "options": " -hgvs -spliceSiteSize 3 -lof -oicr "
        }
        "exomiser": {
            "release": "2109",
            "transcript_source": "refseq",
            "hpo": ["HP:0001156", "HP:0001363", "HP:0011304", "HP:0010055"]
        }
        "hgvs": {
            "full_format": true,
            "use_exon": true
        }
        "options": {
            "append": true
        }
    }
}
```

3.1 parquet

Annotation process using HOWARD Parquet algorithm, for the list of databases available (formats such as Parquet, VCF, TSV, duckDB, JSON).

Examples:

Annotation with multiple databases in multiple formats

```
{
    "parquet": {
        "annotations": {
            "/path/to/database3.parquet": {
                "annotation_fields": {
```

```
                "field1": null,
                "field2": "field2_renamed"
            }
        },
        "/path/to/database4.vcf.gz": {
            "annotation_fields": {
                "INFO": null
            }
        },
        "/path/to/database5.bed.gz": {
            "annotation_fields": {
                "INFO": null
            }
        }
    }
  }
}
```

Annotation with options for a specific database (uniquify values, force append, not force update)

```
{
  "parquet": {
    "annotations": {
      "/path/to/database.bed.gz": {
        "annotation_fields": {
          "INFO": null
        },
        "options": {
          "uniquify": true,
          "annotations_update": false,
          "annotations_append": true
        }
      }
    }
  }
}
```

3.1.1 annotations

Specify a list of databases files available (formats such as Parquet, VCF, TSV, duckDB, JSON).

The section 'annotations_fields' enables users to select specific database fields and optionally rename them (e.g. '"field": null' to keep field name, '"field": "new_name"' to rename field). Use 'INFO' keyword to select all fields within the database INFO/Tags header (e.g. '"INFO": null'). Use 'ALL' keyword to select all fields within the database regardless INFO/Tags header (e.g. '"ALL": null').

If a full path is not provided, the system will automatically detect files within database folders (see Configuration doc) and assembly (see Parameter option).

In addition to select specific fields and optionally rename them, more options are available in 'options' section:

- 'uniquify': to uniquify values or concatenate without aggregation
- 'annotations_update': Update option for annotation. If True, annotation fields will be removed and re-annotated.
- 'annotations_append': Append option for annotation. If True, annotation fields will be annotated only if not annotation exists for the variant.

A keyword 'ALL' can be used to select all available databases, with optional filters on format and release: 'formats:parquet' for all parquet databases, 'formats:parquet' and 'releases:[current]' for all parquet databases in 'current' release (subfolder), 'formats:[parquet,vcf]' and 'releases:[current,devel]' for databases in Parquet or VCF format in 'current' or 'devel' releases).

Examples:

Annotation with dbSNP database with INFO/tags fields, and dbNSFP databases with all fields

```
{
   "annotations": {
      "tests/databases/annotations/current/hg19/avsnp150.parquet": {
         "annotation_fields": {
            "INFO": null
         }
      }
      "tests/databases/annotations/current/hg19/dbnsfp42a.parquet": {
         "annotation_fields": {
            "ALL": null
         }
      }
   }
}
```

Annotation with dbNSFP (only PolyPhen, ClinVar and REVEL score), and rename fields

```
{
   "annotations": {
      "tests/databases/annotations/current/hg19/dbnsfp42a.parquet": {
         "annotation_fields": {
            "Polyphen2_HDIV_pred": "PolyPhen",
            "ClinPred_pred": "ClinVar",
            "REVEL_score": null
         }
      }
   }

}
```

Annotation with refSeq as a BED file

```
{
   "annotations": {
      "tests/databases/annotations/current/hg19/refGene.bed": {
         "annotation_fields": {
            "INFO": null
         }
      }
   }
}
```

Annotation with dbNSFP REVEL annotation (as a VCF file) within configured annotation databases folders (default: '~/howard/databases/annotations/current') and assembly (default: 'hg19')

```
{
   "annotations": {
      "dbnsfp42a.REVEL.vcf.gz": {
         "annotation_fields": {
            "REVEL_score": null,
            "REVEL_rankscore": null
         }
      }
   }

}
```

Annotation with OMIM as a BED file, with uniquification of annotation values (e.g. merge overlaped gene annotations)

```
{
   "annotations": {
      "OMIM.bed.gz": {
```

```
            "annotation_fields": {
               "INFO": null
            },
            "options": {
               "uniquify": true
            }
         }
      }
   }
```

Annotation with all available databases in Parquet for 'current' release

```
{
   "annotations": {
      "ALL": {
         "formats": ["parquet"],
         "releases": ["current"]
      }
   }
}
```

3.2 bcftools

Annotation process using BCFTools. Provide a list of database files and annotation fields.

Examples:

Annotation with multiple databases in multiple formats

```
{
   "parquet": {
      "bcftools": {
         "/path/to/database1.vcf.gz": {
            "annotation_fields": {
               "field1": null,
               "field2": "field2_renamed"
            },
            "options": {
               "header_fields": {
                  "field1": {
                     "Number": ".",
                     "Type": "String",
                     "Description": "field1 as String"
                  }
               }
            }
         },
         "database2.bed.gz": {
            "annotation_fields": {
               "INFO": null
            }
         },
      }
   }
}
```

3.2.1 annotations

Specify the list of database files in formats VCF or BED. Files need to be compressed and indexed.

Section 'annotation_fields' allows to select specific database fields and optionally rename them (e.g. '"field": null' to keep field name, '"field": "new_name"' to rename field). Use 'INFO' or 'ALL' keyword to select all fields within the database

INFO/Tags header (e.g. '"INFO": null', '"ALL": null').

If a full path is not provided, the system will automatically detect files within database folders (see Configuration doc) and assembly (see Parameter option).

The option section provides:

- 'header_fields' allows to override specific fields in the annotation header when updating the database from a VCF file. The keys of the dictionary represent the field names in the annotation header, and the values represent the new values that you want to assign to those fields. This parameter is optional and can be used to customize the annotation header during the update process. If not provided, the default values from the VCF file will be used for the annotation header fields.

Examples:

Annotation with dbSNP database with all fields

```
{
   "annotations": {
      "tests/databases/annotations/current/hg19/avsnp150.vcf.gz": {
         "annotation_fields": {
            "INFO": null
         }
      }
   }
}
```

Annotation with dbNSFP (only PolyPhen, ClinVar and REVEL score), and rename fields and header

```
{
   "annotations": {
      "tests/databases/annotations/current/hg19/dbnsfp42a.vcf.gz": {
         "annotation_fields": {
            "Polyphen2_HDIV_pred": "PolyPhen",
            "ClinPred_pred": "ClinVar",
            "REVEL_score": null
         },
         "options": {
            "Polyphen2_HDIV_pred": {
               "Number": ".",
               "Type": "String",
               "Description": "PolyPhen prediction for Human"
            }
         }
      }
   }
}
```

Annotation with refSeq as a BED file

```
{
   "annotations": {
      "tests/databases/annotations/current/hg19/refGene.bed": {
         "annotation_fields": {
            "INFO": null
         }
      }
   }
}
```

Annotation with dbNSFP REVEL annotation (as a VCF file) within configured annotation databases folders (default: '~/howard/databases/annotations/current') and assembly (default: 'hg19')

```
{
   "annotations": {
```

```
        "dbnsfp42a.REVEL.vcf.gz": {
           "annotation_fields": {
              "REVEL_score": null,
              "REVEL_rankscore": null
           }
        }
     }
  }
```

3.3 annovar

Annotation process using Annovar tool. Provides a list of keywords to select Annovar databases, and defines Annovar options (see Annovar documentation).

This parameter enables users to select specific database using Annovar keyword (e.g. 'refGene', 'clinvar', etc.).

Section 'annotation_fields' allows to select specific database fields and optionally rename them (e.g. '"field": null' to keep field name, '"field": "new_name"' to rename field). Use 'INFO' or 'ALL' keyword to select all fields within the database INFO/Tags header (e.g. '"INFO": null', '"ALL": null').

Section 'options' allows to define Annovar options (e.g. 'protocol', 'operation', 'genebase', 'xref', etc.).

- 'protocol' is the Annovar keyword for the database (e.g. 'refGene', 'clinvar', etc.)
- 'operation' is the Annovar operation (e.g. 'gx', 'g', 'r', 'f'). Default 'f' or autodetected is specific keyword ('g' for 'refGene', 'ensGene' and 'knwonGene', 'r'for 'cytoBand')
- 'genebase' is the Annovar genebase option, using with 'g' operation (e.g. '-splicing_threshold 3 -indel_splicing_threshold 3 -hgvs')
- 'xref' is the Annovar xref option, using with 'gx' operation (e.g. '/path/to/xref/file')
- 'options' is the Annovar additional options (e.g. '-other_options')
- 'header_fields' allows to override specific fields in the annotation header when updating the database from a VCF file. The keys of the dictionary represent the field names in the annotation header, and the values represent the new values that you want to assign to those fields. This parameter is optional and can be used to customize the annotation header during the update process. If not provided, the default values from the VCF file will be used for the annotation header fields.

If a full path is not provided, the system will automatically detect files within database folders (see Configuration doc) and assembly (see Parameter option).

Examples:

Annotation with multiple Annovar databases, with fields selection, and Annovar options

```
{
   "annovar": {
      "annotations": {
         "annovar_annotation1": {
            "annotation_fields": {
               "field1": null,
               "field2": "field2_renamed"
            },
            "options": {
               "protocol": "annovar_keyword1",
               "operation": "gx",
               "genebase": "-splicing_threshold 3 -indel_splicing_threshold 3 -hgvs",
               "arguments": "",
               "xref": "/path/to/xref/file",
               "options": "",
               "header_fields": {
                  "field1": {
                     "Number": ".",
```

```
                    "Type": "String",
                    "Description": "field1 as String"
                },
                "field2_renamed": {
                    "Number": ".",
                    "Type": "String",
                    "Description": "Renamed field2 as String"
                }
            }
        }
    },
    "annovar_annotation2": {
        "annotation_fields": {
            "INFO": null,
            "field2": "field2_renamed"
        },
        "options": {
            "protocol": "annovar_keyword2",
            "operation": "f",
            "genebase": "",
            "arguments": "",
            "xref": "",
            "options": ""
        }
    }
},
"options": {
    "genebase": "",
    "arguments": "",
    "xref": "",
    "options": "",
    "parallelize": null
}
}
}
```

3.3.1 annotations

List of keywords refering to Annovar databases (see Annovar Databases documentation), with a list of selected fields for each of them (rename available)

Examples:

Annotation with ClinVar (fields CLNSIG and CLNDN renamed) and Cosmic (all fields) and header override for ClinVar fields

```
{
   "annotations": {
      "CLINVAR": {
         "annotation_fields": {
            "CLNSIG": "ClinVar_class",
            "CLNDN": "ClinVar_desease"
         },
         "options": {
            "protocol": "clinvar_20221231",
            "operation": "f"
            "header_fields": {
               "ClinVar_class": {
                  "Number": ".",
                  "Type": "String",
                  "Description": "ClinVar classification"
```

```
                },
                "ClinVar_desease": {
                    "Number": ".",
                    "Type": "String",
                    "Description": "ClinVar disease name"
                }
            }
        }
    },
    "cosmic70": {
        "annotation_fields": {
            "INFO": null
        }
    }
  }
}
```

3.3.2 options

List of options available with Annovar tool (see Annovar documentation):

- 'genebase' defines splicing threshold or HGVS annotation with refGene database, which will be used if not defined in the 'annotations' section for a specific database.
- 'arguments' allows to define additional arguments for Annovar command line (e.g. '-hgvs').
- 'xref' allows to define a path to a xref file for Annovar (e.g. '/path/to/xref/file'), if not defined in the 'annotations' section for a specific database.
- 'options' allows to define additional options for Annovar command line (e.g. '-other_options'), for all database annotations.
- 'parallelize' defines parallelization mode for Annovar command line (e.g. 'parallel' to parallelize commands, 'multianno' to use multi annotation Annovar process or null by default).

Examples:

Common options for HGVS Annotation with 'gx' operation (like 'refGene') with a splicing threshold as 3, an additional Gene annotation using external file, and a paralelization mode for Annovar commands

```
{
    "options": {
        "genebase": "-splicing_threshold 3 -indel_splicing_threshold 3",
        "arguments": "-hgvs",
        "xref": "/path/to/xref/file",
        "options": "",
        "parallelize": "parallel"
    }
}
```

3.4 snpeff

Annotation process using snpEff tool and options (see snpEff documentation).

Examples:

Annotation with snpEff databases, with options for HGVS annotation and additional tags.

```
{
    "snpeff": {
        "options": " -hgvs -spliceSiteSize 3 -lof -oicr "
    }
}
```

3.4.1 options

String (as command line) of options available such as:

- filters on variants (regions filter, specific changes as intronic or downstream)
- annotation (e.g. HGVS, loss of function)
- database (e.g. only protein coding transcripts, splice sites size)

Examples:

Annotation with snpEff databases, with options to generate HGVS annotation, specify to not shift variants according to HGVS notation, define splice sites size to 3, add loss of function (LOF), Nonsense mediated decay and OICR tags.

```
{
   "options": " -hgvs -spliceSiteSize 3 -lof -oicr "
}
```

3.4.2 stats

HTML file for snpEff stats. Use keyword 'OUTPUT' to generate file according to output file.

Examples:

Annotation with snpEff databases, and generate a specific stats in HTML format.

```
{
   "stats": "/path/to/stats.html"
}
```

Annotation with snpEff databases, and generate stats in HTML format associated with output file.

```
{
   "stats": "OUTPUT.html"
}
```

3.4.3 csvStats

CSV file for snpEff stats. Use keyword 'OUTPUT' to generate file according to output file.

Examples:

Annotation with snpEff databases, and generate a specific stats in CSV format.

```
{
   "csvStats": "/path/to/stats.csv"
}
```

Annotation with snpEff databases, and generate stats in CSV format associated with output file.

```
{
   "csvStats": "OUTPUT.csv"
}
```

3.5 snpsift

Annotation process using snpSift. Provide a list of database files and annotation fields.

Section 'annotation_fields' allows to select specific database fields and optionally rename them (e.g. '"field": null' to keep field name, '"field": "new_name"' to rename field). Use 'INFO' or 'ALL' keyword to select all fields within the database INFO/Tags header (e.g. '"INFO": null', '"ALL": null').

Examples:

Annotation with multiple databases in multiple formats

```
{
   "snpsift": {
      "annotations": {
         "/path/to/database1.vcf.gz": {
            "annotation_fields": {
               "field1": null,
               "field2": null
            }
            "options": {
               "header_fields": {
                  "field1": {
                     "Number": ".",
                     "Type": "String",
                     "Description": "field1 as String"
                  }
               }
            }
         },
         "/path/to/database2.vcf.gz": {
            "annotation_fields": {
               "field1": null,
               "field2": null
            }
         }
      }
   }
}
```

3.5.1 annotations

Specify the list of database files in formats VCF. Files need to be compressed and indexed.

This parameter enables users to select specific database fields and optionally rename them (e.g. '"field": null' to keep field name, '"field": "new_name"' to rename field). Use 'INFO' or 'ALL' keyword to select all fields within the database INFO/Tags header (e.g. '"INFO": null', '"ALL": null').

If a full path is not provided, the system will automatically detect files within database folders (see Configuration doc) and assembly (see Parameter option).

The option section provides:

- 'header_fields' allows to override specific fields in the annotation header when updating the database from a VCF file. The keys of the dictionary represent the field names in the annotation header, and the values represent the new values that you want to assign to those fields. This parameter is optional and can be used to customize the annotation header during the update process. If not provided, the default values from the VCF file will be used for the annotation header fields.

Examples:

Annotation with dbSNP database with all fields

```
{
   "annotations": {
      "tests/databases/annotations/current/hg19/avsnp150.vcf.gz": {
         "annotation_fields": {
            "INFO": null
         }
      }
   }
}
```

Annotation with dbNSFP (only PolyPhen, ClinVar and REVEL score)

```
{
   "annotations": {
      "tests/databases/annotations/current/hg19/dbnsfp42a.vcf.gz": {
         "annotation_fields": {
            "Polyphen2_HDIV_pred": "PolyPhen",
            "ClinPred_pred": "ClinVar",
            "REVEL_score": null
         },
         "options": {
            "header_fields": {
               "Polyphen2_HDIV_pred": {
                  "Number": ".",
                  "Type": "String",
                  "Description": "PolyPhen prediction for Human"
               }
            }
         }
   }
}
```

Annotation with dbNSFP REVEL annotation (as a VCF file) within configured annotation databases folders (default: '~/howard/databases/annotations/current') and assembly (default: 'hg19')

```
{
   "annotations": {
      "dbnsfp42a.REVEL.vcf.gz": {
         "annotation_fields": {
            "REVEL_score": null,
            "REVEL_rankscore": null
         }
      }
   }
}
```

3.6 bigwig

Annotation process using BigWig files. Provide a list of database files in BigWig format ('.bw') or BigBed format ('.bb') and annotation fields ('annotations_fields' section).

Options can be provided ('options' section) to define the aggregation method (e.g. mean, max, min, sum, coverage join, uniq) to use when multiple values are found for a variant position (such as for indels).

Default methods (global 'options' section) can be defined for all databases (by annotation type), and specific methods can be defined for each database and field (default 'mean' for Integer and Float, 'join' for String).

The option section also provides 'header_fields' option, that allows to override specific fields in the annotation header when updating the database from a VCF file. The keys of the dictionary represent the field names in the annotation header, and the values represent the new values that you want to assign to those fields. This parameter is optional and can be used to customize the annotation header during the update process. If not provided, the default values from the VCF file will be used for the annotation header fields.

Examples:

Annotation with multiple databases in BigWig and BigBed formats, with default and specific aggregation methods.

```
{
   "bigwig": {
      "annotations": {
         "/path/to/database1.bw": {
            "annotation_fields": {
               "field1": null,
               "field2": "field2_renamed"
```

```
            },
            "options": {
               "method": {
                  "field1": "max",
                  "field2_renamed": "min"
               },
               "header_fields": {
                  "field1": {
                     "Number": ".",
                     "Type": "String",
                     "Description": "field1 as String"
                  }
               }
            }
         },
         "/path/to/database2.bb": {
            "annotation_fields": {
               "field1": null,
               "field2": null
            }
         },
      },
      "options": {
        "method": {
           "Integer": "mean",
           "Float": "mean",
           "String": "uniq"
        }
      }
   }
}
```

3.6.1 annotations

Specify the list of database files in BigWig or BigBed format (extension '.bw', '.bb', 'bigwig' or 'bigbed').

The 'annotations_fields' section enables users to select specific database fields and optionally rename them (e.g. '"field": null' to keep field name, '"field": "new_name"' to rename field). Use 'INFO' or 'ALL' keyword to select all fields within the database INFO/Tags header (e.g. '"INFO": null', '"ALL": null').

If a full path is not provided, the system will automatically detect files within database folders (see Configuration doc) and assembly (see Parameter option).

A URL can be provided as a database file (experimental). In this case, associated header file will be automatically generated with a uniq value as the name of the file (cleaned for avoid special characters, and '.bw' extension).

Examples:

Annotation with GERP database with all fields

```
{
   "annotations": {
      "tests/databases/annotations/current/hg19/gerp.bw": {
         "annotation_fields": {
            "INFO": null
         }
      }
   }
}
```

Annotation with GERP scores renamed and a specific aggregation method 'max' for the 'GERP_score' field

```
{
   "annotations": {
      "tests/databases/annotations/current/hg19/gerp.bw": {
         "annotation_fields": {
            "gerp": "GERP_score"
         },
         "options": {
            "method": {
               "GERP_score": "max"
            },
            "header_fields": {
               "GERP_score": {
                  "Number": ".",
                  "Type": "Float",
                  "Description": "GERP_score as Float"
               }
            }
         }
      }
   }
}
```

Annotation with GERP from a distante database (experimental)

```
{
   "annotations": {
      "https://hgdownload.soe.ucsc.edu/gbdb/hg19/bbi/All_hg19_RS.bw": {
         "annotation_fields": {
            "INFO": null
         }
      }
   }
}
```

3.6.2 options

Parameters for BigWig/BigBed annotation, including default aggregation methods and specific aggregation methods for each database and field.

Aggregation methods available: 'mean', 'max', 'min', 'sum', 'coverage' for Integer or Float value type, and 'join', 'uniq' for String value type.

By default, 'mean' method is used for Integer and Float value type, and 'join' method for String value type.

Default and specific aggregation methods can be combined.

Examples:

Parameters to define default aggregation methods by type

```
{
   "options": {
      "method": {
         "Integer": "mean",
         "Float": "mean",
         "String": "uniq"
      }
   }
}
```

3.7 exomiser

Annotation process using Exomiser tool and options (see Exomiser website documentation).

Examples:

Annotation with Exomiser, using database release '2109', transcripts source as UCSC and a list of HPO terms.

```
{
   "exomiser": {
      "release": "2109"
      "transcript_source": "refseq"
      "hpo": ["HP:0001156", "HP:0001363", "HP:0011304", "HP:0010055"]
   }
}
```

3.7.1 release

Release of Exomiser database. This option replace the 'release' variable in 'application.properties' file (see 'exomiser_application_properties' option). The release will be downloaded if it is not available locally.

Examples:

Annotation with release '2109' of Exomiser database.

```
{
   "release": "2109"
}
```

3.7.2 transcript_source

Transcription source of Exomiser. This option replace the 'transcript_source' variable in 'application.properties' file (see 'exomiser_application_properties' option). The release will be downloaded if it is not available locally.

Examples:

Annotation with transcription source 'refseq' of Exomiser.

```
{
   "transcript_source": "refseq"
}
```

3.7.3 hpo

List of HPO for Exomiser. This option replace the 'hpo' variable in 'application.properties' file (see 'exomiser_application_properties' option). The release will be downloaded if it is not available locally.

Examples:

Annotation with a list of 4 HPO for Exomiser.

```
{
   "hpo": ["HP:0001156", "HP:0001363", "HP:0011304", "HP:0010055"]
}
```

3.8 splice

Annotation process using Splice tool and options. This annotation will be proccessed only for variants that are not already annotated (i.e. without annotation like 'SpliceAI_*' and 'SPiP_*')

Examples:

Annotation with Splice, using database splice mode ('one'), spliceAI distance (500) and spliceAI mask (1).

```
{
   "splice": {
      "split_mode": "one",
      "spliceai_distance": 500,
      "spliceai_mask": 1
   }
}
```

3.8.1 split_mode

Split mode of Exomiser database (default 'one'):

- all: report all annotated transcript for one gene.
- one: keep only the transcript with the most pathogenic score (in case of identical score, take the first).
- list: keep transcript provided in transcript file, if no matching transcript in file 'one' mode is activated.
- mixed: 'one' mode, if identical score, list mode is activated.

Examples:

Split mode to report all annotated transcript for one gene.

```
{
   "split_mode": "all"
}
```

3.8.2 spliceai_distance

Maximum distance between the variant and gained/lost splice site (default: 500).

Examples:

Maximum distance of '500' between variant and splice site.

```
{
   "spliceai_distance": 500
}
```

3.8.3 spliceai_mask

Mask scores representing annotated acceptor/donor gain and unannotated acceptor/donor loss (default: 1).

Examples:

Mask score of '1' for acceptor/donor gain fain and loss.

```
{
   "spliceai_mask": 1
}
```

3.8.4 transcript

Path to a list of transcripts of preference (default '').

Examples:

Path to file of transcripts.

```
{
   "transcript": "tests/data/transcripts.tsv"
}
```

3.8.5 rm_snps

Do not consider SNV for the analysis, only Indels and MNV (default 'false').

Examples:

Analysing only non SNV.

```
{
   "rm_snps": "true"
}
```

3.8.6 rm_annot

Remove existing annotation before analysing (default 'true').

Examples:

Remove annotation before analysing.

```
{
   "rm_annot": "true"
}
```

3.8.7 whitespace

Remove spaces in INFO field, 'true' to remove (default 'true').

Examples:

Remove spaces in INFO field.

```
{
   "whitespace": "true"
}
```

3.9 docker

Annotation process using Docker containers. This section defines one or multiple Docker entries to run annotation tools in containers.

This parameter file is dedicated to run-specific settings. Structural Docker settings are configured in the configuration file under config.tools..docker (annotation/runtime).

Each entry exports variants to an input VCF, runs a container command with mounted resources, and imports output VCF annotations into the variants table.

Examples:

Docker-based annotation with one VEP entry (run-specific parameters)

```
{
   "docker": {
     "entries": {
       "vep": {
         "tool": "vep",
         "resources": {
           "threads": 4,
           "memory": "8G"
         },
         "parameters": [
           "--everything",
           "--cache",
           { "key": "--assembly", "value": "GRCh37" }
         ],
         "where_clause": " WHERE \"#CHROM\" = 'chr7' "
       }
     }
   }
}
```

3.9.1 entries

Dictionary of Docker annotation entries. Each key is an entry name and each value is an entry definition.

Examples:

Two Docker annotation entries

```
{
  "entries": {
    "vep": { "tool": "vep" },
    "custom": { "tool": "mytool" }
  }
}
```

3.9.1.1 entry_name Docker entry identifier (JSON object key).

One entry generally maps to one containerized annotation command.

Examples:

Entry named 'vep'

```
{
  "vep": {
    "tool": "vep"
  }
}
```

3.9.1.1.1 tool Name of the tool configuration to use from configuration section 'tools' (e.g. tools..docker.image).

Examples:

Use tool definition 'vep' from configuration

```
{
  "tool": "vep"
}
```

3.9.1.1.2 resources Optional entry-level resources override for this run.

Threads and memory are automatically capped to available resources (run limits and system availability) to avoid invalid container commands.

Examples:

Resource override values

```
{
  "resources": {
    "threads": 4,
    "memory": "8G"
  }
}
```

threads

Threads value for this entry.

Type: `int`

Default: `-1`

Examples:

```
{
  "threads": 4
}
```

memory

Memory value for this entry (string or number, depending on tool syntax).

Type: `str`

Format: `FLOAT[kMG]`

Default: `None`

Examples:

```
{
   "memory": "8G"
}
```

3.9.1.1.3 parameters List of run-specific command parameters appended to the executable.

Supported formats include string flags, token arrays, or key/value objects.

These values are merged with default parameters configured in config.tools..docker.annotation.defaults.parameters.

Examples:

```
{
   "parameters": [
     "--cache",
     ["--species", "homo_sapiens"],
     { "key": "--assembly", "value": "GRCh37" }
   ]
}
```

3.9.1.1.4 options Additional run-specific command options merged with 'parameters'.

Both defaults and entry values are merged; if the same CLI key appears multiple times (e.g. --compress_output), entry value overrides default value.

Examples:

```
{
   "options": [
     "--flag"
   ]
}
```

3.9.1.1.5 where_clause Optional SQL WHERE clause to select variants before export for this entry.

A pre-check query with LIMIT 1 is executed; when no variant matches, entry execution is skipped.

Examples:

```
{
   "where_clause": "#CHROM = '1' AND POS >= 100000"
}
```

3.9.1.1.6 output_pattern Optional glob pattern override used to locate output VCF when output path does not directly exist.

If omitted, default value from config.tools..docker.annotation.defaults.output_pattern is used when defined.

Examples:

```
{
   "output_pattern": "*.vcf.gz"
}
```

3.10 hgvs

HOWARD annotates variants with HGVS annotation using HUGO HGVS internation Sequence Variant Nomenclature (http://varnomen.hgvs.org/). Annotation refere to refGene and genome to generate HGVS nomenclature for all available transcripts. This annotation add 'hgvs' field into VCF INFO column of a VCF file. Several options are available, to add gene, exon and protein information, to generate a 'full format' detailed annotation, to choose codon format.

Examples:

> HGVS annotation with operations for generate variant_id and variant type, extract HGVS from snpEff annotation, select NOMEN from snpEff HGVS with a list of transcripts of preference
>
> ```
> {
>    "hgvs": {
>      "full_format": true,
>      "use_exon": true
>    }
> }
> ```

3.10.1 use_gene

Add Gene information to generate HGVS annotation (e.g. 'NM_152232**(TAS1R2)**:c.231T>C').

Default: `False`

Examples:

> Use Gene in HGVS annotation
>
> ```
> {
>    "use_gene": true
> }
> ```

3.10.2 use_exon

Add Exon information to generate HGVS annotation (e.g. 'NM_152232(exon2):c.231T>C'). Only if 'use_gene' is not enabled.

Default: `False`

Examples:

> Use Exon in HGVS annotation
>
> ```
> {
>    "use_exon": true
> }
> ```

3.10.3 use_protein

Use Protein level to generate HGVS annotation (e.g. 'NP_689418:p.Cys77Arg'). Can be used with 'use_exon' or 'use_gene'.

Default: `False`

Examples:

> Use Protein in HGVS annotation
>
> ```
> {
>    "use_protein": true
> }
> ```

3.10.4 add_protein

Add Protein level to DNA HGVS annotation (e.g. 'NM_152232:c.231T>C,NP_689418:p.Cys77Arg').

Default: `False`

Examples:

Add Protein level to DNA HGVS annotation

```
{
   "add_protein": true
}
```

3.10.5 full_format

Generates HGVS annotation in a full format (non-standard, e.g. 'TAS1R2:NM_152232:NP_689418:c.231T>C:p.Cys77Arg', 'TAS1R2:NM_152232:NP_689418:exon2:c.231T>C:p.Cys77Arg'). Full format use all information to generates an exhaustive annotation. Use specifically 'use_exon' to add exon information.

Default: `False`

Examples:

Use full format for HGVS annotation

```
{
   "full_format": true
}
```

3.10.6 codon_type

Amino Acide Codon format type to use to generate HGVS annotation (default '3'):

- '1': codon in 1 character (e.g. 'C', 'R')
- '3': codon in 3 character (e.g. 'Cys', 'Arg')
- 'FULL': codon in full name (e.g. 'Cysteine', 'Arginine')

Type: `str`

Choices: `['1', '3', 'FULL']`

Default: `3`

Examples:

Amino Acide Codon format with 1 character

```
{
   "codon_type": "1"
}
```

Amino Acide Codon format with 3 character

```
{
   "codon_type": "3"
}
```

Amino Acide Codon format with full name

```
{
   "codon_type": "FULL"
}
```

3.10.7 refgene

Path to refGene annotation file (see HOWARD User Guide).

Type: `Path`

Default: `None`

Examples:

Path to refSeq file

```
{
   "refgene": "~/howard/databases/refseq/current/hg19/ncbiRefSeq.txt"
}
```

3.10.8 refseqlink

Path to refGeneLink annotation file (see HOWARD User Guide).

Type: `Path`

Default: `None`

Examples:

Path to refSeq file

```
{
   "refseqlink": "~/howard/databases/refseq/current/hg19/ncbiRefSeqLink.txt"
}
```

4 calculation

Calculation process operations that are defiend in a Calculation Configuration JSON file. List available calculation operations with possible options (see Calculation JSON file help).

Examples:

Calculation of operations 'operation1' and 'operation2' (with options) defined in 'calculation_config.json' file

```
{
   "calculation": {
     "calculations": {
       "operation1": null,
       "operation2": {
         "options": {
           "option1": "value1",
           "option2": "value2"
         }
       }
     },
     "calculation_config": "calculation_config.json"
   }
}
```

4.1 calculations

List of operations to process with possible options (see Calculation JSON file help).

Examples:

Calculation with operations for generate variant_id and variant type, extract HGVS from snpEff annotation, calculate number of samples and list of samples for each variant, select NOMEN from snpEff HGVS with a prioritized transcript (from prioritization transcript calculation) and list of transcripts of preference, a list of NOMEN fields, with two specific NOMEN patterns

```
{
   "calculations": {
     "variant_id": null,
     "vartype": null,
     "snpeff_hgvs": null,
     "find_samples": {
       "tags": {
         "count_samples": "count",
         "list_samples": "list"
```

```
        }
      },
      "NOMEN": {
        "options": {
          "hgvs_field": "snpeff_hgvs",
          "hgvs_fields": ["snpeff_hgvs"],
          "uniquify_hgvs": false,
          "transcripts": "tests/data/transcripts.tsv",
          "transcripts_table": "variants",
          "transcripts_column": "PZTTranscript",
          "transcripts_order", ["column", "file"],
          "fields": ["GNOMEN", "TVNOMEN", "ENOMEN", "CNOMEN"],
          "pattern": {
            "NOMEN": "GNOMEN:TNOMEN:ENOMEN:CNOMEN:RNOMEN:NNOMEN:PNOMEN",
            "NOMENO": "TNOMEN(ENOMEN):CNOMEN:RNOMEN:NNOMEN"
          },
        }
      }
    }
}
```

4.2 calculation_config

Calculation configuration JSON file.

Type: `Path`

Default: `None`

Examples:

Calculation configuration JSON file as an option

```
{
   "calculation_config": "calculation_config.json"
}
```

5 prioritization

Prioritization process use a JSON configuration file that defines all profiles that can be used. By default, all profiles will be calculated from the JSON configuration file, and the first profile will be considered as default. Proritization annotations (INFO/tags) will be generated depending of a input list (default 'PZScore' and 'PZFlag'), for all profiles (e.g. 'PZScore_GERMLINE' for 'GERMLINE' profile) and for default profile (e.g. 'PZScore' for default). Prioritization score mode is 'HOWARD' by default.

Examples:

Prioritization with 'GENOME' and 'GERMLINE' (default) profiles, from a list of configured profiles, only 3 prioritization fields returned, and score calculated in 'VaRank' mode

```
{
   "prioritization": {
     "prioritizations": "config/prioritization_profiles.json",
     "profiles": ["GENOME", "GERMLINE"],
     "default_profile": "GERMLINE",
     "pzfields": ["PZScore", "PZFlag", "PZComment"],
     "prioritization_score_mode": "VaRank"
   }
}
```

5.1 prioritizations

Prioritization configuration profiles JSON file defining profiles to calculate. All configured profiles will be calculated by default (see 'profiles' parameter). First profile will be considered as 'default' if none are provided (see 'default_profile' parameter). Default score calculation mode is 'HOWARD'. This option refers to the quick prioritization command line parameter `--prioritizations`.

Type: `str`

Default: `None`

Examples:

Prioritization configuration profile JSON file

```
{
   "prioritizations": "config/prioritization_profiles.json"
}
```

5.2 profiles

Prioritization profiles to consider, from the list of configured profiles. If empty, all configured profiles will be calculated. First profile will be considered as 'default' if none are provided (see 'default_profile' parameter). Prioritization annotations (INFO/tags) will be generated for all these profiles (e.g. 'PZScore_GERMLINE' for 'GERMLINE' profile).

Type: `str`

Default: `None`

Examples:

Prioritization with 'GERMLINE' profile only

```
{
   "profiles": ["GERMLINE"]
}
```

Prioritization with 'GENOME' and 'GERMLINE' profiles

```
{
   "profiles": ["GENOME", "GERMLINE"]
}
```

5.3 default_profile

Prioritization default profile from the list of processed profiles. Prioritization annotations (INFO/tags) will be generated for this default profile (e.g. 'PZScore', 'PZFlags').

Type: `str`

Default: `None`

Examples:

Prioritization default profile 'GERMLINE'

```
{
   "default_profile": "GERMLINE"
}
```

5.4 pzfields

Prioritization annotations (INFO/tags) to generate. By default 'PZScore', 'PZFlags'.

Prioritization fields can be selected from:

- PZScore: calculated score from all passing filters, depending of the mode
- PZFlag: final flag ('PASS' or 'FILTERED'), with strategy that consider a variant is filtered as soon as at least one filter do not pass. By default, the variant is considered as 'PASS' (no filter pass)

- PZComment: concatenation of all passing filter comments
- PZTags: combinason of score, flags and comments in a tags format (e.g. 'PZFlag#PASS|PZScore#15|PZComment#Described on ...')
- PZInfos: information about passing filter criteria

Type: `str`

Default: `PZScore,PZFlag`

Examples:

Prioritization annotations (INFO/tags) list

```
{
   "pzfields": ["PZScore", "PZFlag", "PZComment"]
}
```

5.5 prioritization_score_mode

Prioritization score can be calculated following multiple mode. The HOWARD mode will increment scores of all passing filters (default). The VaRank mode will select the maximum score from all passing filters.

Type: `str`

Choices: `['HOWARD', 'VaRank']`

Default: `HOWARD`

Examples:

Prioritization score calculation mode 'HOWARD'

```
{
   "prioritization_score_mode": "HOWARD"
}
```

Prioritization score calculation mode 'VaRank'

```
{
   "prioritization_score_mode": "VaRank"
}
```

5.6 pzprefix

Prioritization prefix for all annotations generated by prioritization.

Type: `str`

Default: `PZ`

Examples:

Prioritization prefix by default ('PZ'):

```
{
   "pzprefix": "PZ"
}
```

Specific prioritization prefix:

```
{
   "pzprefix": "PrioritiZation_"
}
```

Specific prioritization prefix for transcript (see below):

```
{
   "pzprefix": "PZT"
}
```

6 transcripts

Transcripts information to create transcript view. Useful to add transcripts annotations in INFO field, to calculate transcripts specific scores (see Calculation JSON file help), to merge and map transcript IDs (e.g. from Ensembl to refSeq), or prioritize transcripts (see Priorization JSON file help).

Type: `dict`

Default: `{}`

Examples:

Trancripts information from snpEff and dbNSFP annotation

```
{
   "transcripts": {
     "table": "transcripts",
     "transcripts_info_field_json": "transcripts_json",
     "transcripts_info_field_format": "transcripts_ann",
     "transcripts_info_json": "transcripts_json",
     "transcripts_info_format": "transcripts_format",
     "transcript_id_remove_version": true,
     "transcript_id_mapping_file": "transcripts.for_mapping.tsv",
     "transcript_id_mapping_force": false,
     "struct": {
         "from_column_format": [
             {
                 "transcripts_column": "ANN",
                 "transcripts_infos_column": "Feature_ID",
                 "column_clean": true,
                 "column_case": "lower"
             }
         ],
         "from_columns_map": [
             {
                 "transcripts_column": "Ensembl_transcriptid",
                 "transcripts_infos_columns": [
                     "genename",
                     "Ensembl_geneid",
                     "LIST_S2_score",
                     "LIST_S2_pred"
                 ],
                 "column_rename": {
                     "LIST_S2_score": "LISTScore",
                     "LIST_S2_pred": "LISTPred"
                 },
             },
             {
                 "transcripts_column": "Ensembl_transcriptid",
                 "transcripts_infos_columns": [
                     "genename",
                     "VARITY_R_score",
                     "Aloft_pred"
                 ]
             }
         ]
     },
     "prioritization": {
        "profiles": ["transcripts"],
        "prioritization_config": "config/prioritization_transcripts_profiles.json",
        "pzprefix": "PZT",
```

```
            "pzfields": ["Score", "Flag", "LISTScore", "LISTPred"],
            "prioritization_transcripts_order": {
                 "PZTFlag": "DESC",
                 "PZTScore": "DESC"
            }
            "prioritization_score_mode": "HOWARD",
            "prioritization_transcripts_file": null,
            "prioritization_transcripts_columns": null,
            "prioritization_transcripts_force": false,
            "prioritization_transcripts_version_force": false
         },
         "export": {
            "output": "/tmp/output.tsv.gz"
         }
      }
   }
```

6.1 table

Transcripts table name to create.

Type: `str`

Default: `transcripts`

Examples:

Name of transcript table:

```
{
   "table": "transcripts"
}
```

6.2 transcripts_info_field_json

Transcripts INFO field name to add in VCF INFO field in JSON format.

Type: `str`

Default: `None`

Examples:

Transcripts INFO field name:

```
{
   "transcripts_info_field_json": "transcripts_json"
}
```

6.3 transcripts_info_field_format

Transcripts INFO field name to add in VCF INFO field in strutured format.

Type: `str`

Default: `None`

Examples:

Transcripts INFO field name:

```
{
   "transcripts_info_field_format": "transcripts_ann"
}
```

6.4 transcripts_info_json

Transcripts column name to add to transcripts table in JSON format.

Type: `str`

Default: `None`

Examples:

Transcripts column name:

```
{
   "transcripts_info_json": "transcripts_json"
}
```

6.5 transcripts_info_format

Transcripts column name to add to transcripts table in structured format.

Type: `str`

Default: `None`

Examples:

Transcripts column name:

```
{
   "transcripts_info_format": "transcripts_format"
}
```

6.6 transcript_id_remove_version

When merging and mapping transcript IDs, remove possible version of transcript (e.g 'NM_123456.2' to 'NM_123456').

Type: `Boolean`

Default: `False`

Examples:

Remove transcript version when merging and mapping:

```
{
   "transcript_id_remove_version": true
}
```

6.7 transcript_id_mapping_file

When merging and mapping transcript IDs, indicate a transcript mapping file that provides mapping between transcripts IDs (useful to map refSeq and Ensembl transcript IDs).

Type: `Path`

Default: `None`

Examples:

Transcript IDs mapping file:

```
{
   "transcript_id_mapping_file": "My_transcripts_mapping_file.tsv.gz"
}
```

6.8 Example of transcript ID mapping file

Transcript IDs mapping file is a tab-delimited file with 2 columns:

- first column corresponds to the reference transcript ID to use
- second column correspond to an alias of the reference transcript

Second column can be empty (no alias is provided). Transcript IDs can include version or not (see transcript_id_remove_version section)

Examples:

Example of transcripts mapping file:

```
NM_001005484    ENST00000641515.1
NM_005228.5     ENST00000275493.7
NM_001346897    ENSG00000146648.21
NM_001346941
NM_005228
```

6.9 transcript_id_mapping_force

When merging and mapping transcript IDs, allows to filter transcript IDs only if they are present in first column of the transcript mapping file (see transcript_id_mapping_file section).

Type: `Boolean`

Default: `False`

Examples:

Filter transcripts IDs only if present in mapping file:

```
{
   "transcript_id_mapping_force": true
}
```

6.10 struct

Structure of transcripts information, corresponding to columns dedicated to transcripts, such as:

- 'from_column_format': a uniq annotation field with a specific format, like snpEff annotation,
- 'from_columns_map': a list of annotation fields corresponding to transcripts in another specific field, like dbNSFP annotation.

Some parameters are commons between these structure (e.g. 'column_rename', 'column_clean' and 'column_case').

Type: `dict`

Default: `{}`

6.10.1 from_column_format

Structure of transcripts information from a uniq annotation field with a specific format (such as snpEff annotation):

- 'transcripts_column' correspond to INFO field with annotations
- 'transcripts_infos_column' correspond to transcription ID annotations field within INFO field

Column can be renamed, cleaned and/or case changed (see below).

Type: `dict`

Default: `{}`

Examples:

Structure from snpEff annotation (columns names must be clean or changed for standard snpEff annotations):

```
{
   "from_column_format": [
     {
       "transcripts_column": "ANN",
       "transcripts_infos_column": "Feature_ID",
       "column_rename": null,
       "column_clean": true,
       "column_case": null
     }
   ]
}
```

6.10.2 from_columns_map

list of annotation fields corresponding to transcripts in another specific field (such as dbNSFP annotation):

- 'transcripts_column' correspond to INFO field with transcription ID
- 'transcripts_infos_columns' correspond to a list of INFO fields with transcript information

Column can be renamed, cleaned and/or case changed (see below).

Type: `dict`

Default: `{}`

Examples:

Structure from dbNSFP annotations (with 2 columns renamed):

```
{
   "from_columns_map": [
     {
       "transcripts_column": "Ensembl_transcriptid",
       "transcripts_infos_columns": [
         "genename",
         "Ensembl_geneid",
         "LIST_S2_score",
         "LIST_S2_pred"
       ],
       "column_rename": {
         "LIST_S2_score": "LISTScore",
         "LIST_S2_pred": "LISTPred"
       },
       "column_clean": false,
       "column_case": null
     }
   ]
}
```

6.10.3 from_variants

List of fields from variants annotations, either from 'INFO' column or a specific column already available in the table.

- 'fields' is a list of fields to include
- 'prefix' is used to add a prefix on fields
- 'INFO' enables full annotation inclusion by adding 'INFO' column

Type: `dict`

Default: `{}`

Examples:

List of fields ('CLNSIG' and 'gnomad') to include without prefix:

```
{
   "from_variants": {
      "fields": ["CLNSIG", "gnomad"],
      "prefix": "",
      "INFO": false,
   },
}
```

6.10.4 commons parameters

Some parameters are commons between these structure:

- 'column_rename': dict defining mapping of column name changes
- 'column_clean': if true, clean column name to remove not alphanum characters not allowed in VCF (e.g. space, dash)
- 'column_case': rename column into lowercase ('lower') or uppercase ('upper')

Combining 'column_clean' and 'column_case' ensure well formed VCF field name and merging identical columns (e.g. same field 'Gene_Name', 'Gene name' and 'genename' from multiple sources). However, controling column names through 'column_rename' is much more efficient.

Examples:

Commons parameters by default:

```
{
  "column_rename": null,
  "column_clean": false,
  "column_case": null
}
```

Parameters to change 2 column names:

```
{
  "column_rename": {
    "LIST_S2_score": "LISTScore",
    "LIST_S2_pred": "LISTPred"
  },
  "column_clean": false,
  "column_case": null
}
```

Parameters to ensure well-named column from format annotations field (such as snpEff):

```
{
  "column_rename": null,
  "column_clean": true,
  "column_case": null
}
```

6.11 prioritization

Prioritization parameters for transcripts (see Prioritization section and Priorization JSON file help), defining prioritization criteria with a configuration file of all available profiles and which profiles to use, which prioritization method to use (e.g. 'HOWARD', 'VaRank').

Prioritized transcripts fields can be defined to provide VCF fields specific to the choosen transcript, using a specific prefix (e.g. 'PZTScore', 'PZTFlag'). The selected 'best' transcript ID is always provided (e.g. 'PZTTranscript'). Extra annotation fields can also be defined (e.g. 'LISTScore', 'LISTPred'), as well as prioritizations informations from multiple profiles (e.g. 'PZTScore_myprofile', 'PZTScore_myotherprofile' in 'pzfields' section with 2 profiles 'myprofile' and 'myotherprofile' in 'profiles' section).

In order to choose the 'best' transcript, parameteres can define order of transcripts (by annotation columns or a preference transcripts file), by dealing with transcript versions.

Type: `dict`

Default: `{}`

Examples:

Prioritization of transcripts in 'HOWARD' mode with 'transcripts' profiles available in a configuration JSON file, with 'PZT' as prefix:

```
{
   "prioritization": {
      "profiles": ["transcripts"],
      "default_profile": "transcripts",
      "prioritization_config": "config/prioritization_transcripts_profiles.json",
      "prioritization_score_mode": "HOWARD",
      "pzprefix": "PZT",
      "pzfields": ["Score", "Flag", "LISTScore", "LISTPred"],
      "prioritization_transcripts_order": {
         "PZTFlag": "DESC",
         "PZTScore": "DESC"
      },
      "prioritization_transcripts_file": null,
      "prioritization_transcripts_columns": null,
      "prioritization_transcripts_force": false,
      "prioritization_transcripts_version_force": false
   }
}
```

6.11.1 profiles

See Prioritization section

Type: `str`

Default: `None`

6.11.2 default_profile

See Prioritization section

Type: `str`

Default: `None`

6.11.3 prioritization_config

See Prioritization section

Type: `Path`

Default: `None`

Examples:

Prioritization configuration JSON file as an option

```
{
   "prioritization_config": "prioritization_config.json"
}
```

6.11.4 prioritization_score_mode

See Prioritization section

Type: `str`

Choices: `['HOWARD', 'VaRank']`

Default: `HOWARD`

6.11.5 pzprefix

See Prioritization section

6.11.6 pzfields

See Prioritization section

Type: `str`

Default: `PZScore,PZFlag`

6.11.7 prioritization_transcripts_order

Defines the order of transcripts to determine which one is chosen (by default PZTFlag and PZTScore). All available annotation can be used (e.g. scores, length of transcript, predictions...). The first transcript will be choosen in case of equal order.

Type: `dict`

Default: `{}`

Examples:

Default order of transcript using Flag (PASS before FILTERED) and Score (higher scores before):

```
{
   "prioritization_transcripts_order": {
      "PZTFlag": "DESC",
      "PZTScore": "DESC"
   }
}
```

Order of transcript using Flag, Score and additional specific spliceAI_score:

```
{
   "prioritization_transcripts_order": {
      "PZTFlag": "DESC",
      "PZTScore": "DESC",
      "spliceAI_score": "DESC"
   }
}
```

6.11.8 prioritization_transcripts_file

Defines a file with an ordered list of transcripts of preference. The first transcript in this list will be chosen, if no order is define (see above), or if this list is not forced (see below).

Type: `Path`

Default: `None`

Examples:

File with transcripts of preference:

```
{
   "prioritization_transcripts_file": "transcripts.tsv"
}
```

6.11.9 prioritization_transcripts_columns

Defines the columns to use as allowed transcripts from the transcripts table for prioritization. This option filter transcripts that are not not present in these columns. Useful to ensure transcripts are present in HGVS annotation for example, or are at least annotated by some transcript annotation sources.

Type: `Path`

Default: `None`

Examples:

Columns that contains transcripts allowed for prioritization (e.g. 'FeatureID' from snpEff HGVS annotation, 'DBNSFP_transcript' from dbNSFP annotation):

```
{
   "prioritization_transcripts_columns": ["FeatureID", "DBNSFP_transcript"]
}
```

6.11.10 prioritization_transcripts_force

Force to use the list of transcripts of preference define in the provided file (see above).

Type: `Boolean`

Default: `False`

Examples:

Force using transcripts of preference in file:

```
{
   "prioritization_transcripts_force": true
}
```

6.11.11 prioritization_transcripts_version_force

By default, versions of transcripts are not considered when comparison is needed (e.g. transcript ID and list of transcript of preference). If true, all transcript ID will be considered with their version.

Type: `Boolean`

Default: `False`

Examples:

Force using transcripts version:

```
{
   "prioritization_transcripts_version_force": true
}
```

6.12 export

Options to export transcripts view/table into a file ('output' parameter). All HOWARD format are available (e.g. VCF, Parquet, TSV). For VCF format, all columns will be concatenate into INFO column, otherwise each column will be exported.

Type: `Dict`

Default: `{}`

Examples:

Export as compressed TSV, including header:

```
{
   "export": {
     "output": "/tmp/output.tsv.gz",
     "header_in_output": true
   }
}
```

Export as VCF:

```
{
   "export": {
     "output": "/tmp/output.vcf"
   }
}
```

Export as Parquet, generate header file and include INFO column:

```
{
   "export": {
     "output": "/tmp/output.parquet",
     "export_header": true,
     "add_info": true
   }
}
```

6.12.1 output

Output file to export transcripts view/table. The output file format is deduced from the file extension (e.g. '.vcf', '.parquet', '.tsv.gz').

Type: `Path`

Default: `None`

Examples:

Output file to export transcripts view/table:

```
{
   "output": "/tmp/output.tsv.gz"
}
```

6.12.2 export_header

Export header file ('.hdr') corresoonding to output file. By default, header file is not generated.

Type: `Boolean`

Default: `True`

Examples:

Include header in output file:

```
{
   "export_header": true
}
```

Do not include header in output file:

```
{
   "export_header": false
}
```

6.12.3 header_in_output

Include header in output file. By default, header is not included in output file (excpet for VCF format).

Type: `Boolean`

Default: `True`

Examples:

Include header in output file:

```
{
    "header_in_output": true
}
```

Do not include header in output file:

```
{
    "header_in_output": false
}
```

6.12.4 add_info

Add INFO field to output file. By default, INFO field is not added to output file (except for VCF fomat).

Type: `Boolean`

Default: `False`

Examples:

Add INFO field to output file:

```
{
    "add_info": true
}
```

Do not add INFO field to output file:

```
{
    "add_info": false
}
```

6.13 explode

Explode options for INFO/tags annotations within output file.

6.13.1 explode_infos

Explode VCF INFO/Tag into table columns (e.g. 'variants', 'transcripts').

Default: `False`

6.13.2 explode_infos_prefix

Explode VCF INFO/Tag with a specific prefix.

Type: `str`

Default:

6.13.3 explode_infos_fields

Explode VCF INFO/Tag specific fields/tags. Keyword `*` specify all available fields, except those already specified. Pattern (regex) can be used, such as `.*_score` for fields named with '_score' at the end. Examples:

- 'HGVS,SIFT,Clinvar' (list of 3 fields)
- 'HGVS,*,Clinvar' (list of 2 fields with all other fields in the middle)
- 'HGVS,.*_score,Clinvar' (list of 2 fields with all scores in the middle)
- 'HGVS,.*_score,*' (1 field, scores, all other fields at the end)

Type: `str`

Default: `*`

7 stats

Statistics on loaded variants.

7.1 stats_md

Stats Output file in MarkDown format.

Type: `Path`

Default: `None`

Examples:

Export statistics in Markdown format

```
{
   "stats_md": "/tmp/stats.pdf"
}
```

7.2 stats_json

Stats Output file in JSON format.

Type: `Path`

Default: `None`

Examples:

Export statistics in JSON format

```
{
   "stats_json": "/tmp/stats.json"
}
```

7.3 stats_html

Stats Output file in HTML format.

Type: `Path`

Default: `None`

Examples:

Export statistics in JSON format

```
{
   "stats_html": "/tmp/stats.html"
}
```

7.4 stats_pdf

Stats Output file in PDF format.

Type: `Path`

Default: `None`

Examples:

Export statistics in JSON format

```
{
   "stats_pdf": "/tmp/stats.pdf"
}
```

7.5 annotations_stats

Add statistics on annotations (INFO/tags)).

Default: `False`

7.6 queries

Queries to add on statistics.

Beware that queries are executed on the 'variants_view' view by default. If you want to use another view, please specify it with the 'queries_view' parameter.

Moreover, query limit is suggested to avoid long processing time and huge output files.

Type: `dict`

Default: `None`

Examples:

Queries to add on statistics:

```
{
   "queries": {
     "First 10 variants": "SELECT \"#CHROM\", POS, REF, ALT FROM variants_view LIMIT 10",
     "First 10 INFO tags": "SELECT INFOS.* FROM variants_view LIMIT 10",
   }
}
```

7.7 queries_view

Variants view name to use with queries to add on statistics. By default, the 'variants_view' view is used.

Type: `str`

Default: `None`

Examples:

Variants view name for stats queries:

```
{
   "queries_view": "variants_view_for_stats_queries"
}
```

8 query

Query options tools. Mainly a SQL query, based on 'variants' table corresponding on input file data, or a independant query. Print options for 'query' tool allow limiting number of lines and choose printing mode.

Type: `str`

Default: `None`

8.1 query

Query in SQL format (e.g. 'SELECT * FROM variants LIMIT 50').

Type: `str`

Default: `None`

Examples:

Simple query to show all variants file

```
SELECT "#CHROM", POS, REF, ALT, INFO
FROM variants
```

8.2 query_limit

Limit of number of row for query (only for print result, not output).

Type: `int`

Default: `10`

8.3 query_print_mode

Print mode of query result (only for print result, not output). Either None (default), 'dataframe', 'markdown', 'tabulate' or disabled. If None, print mode is 'dataframe' if no export file is provided.

Type: `str`

Choices: `[None, 'dataframe', 'markdown', 'tabulate', 'disabled']`

Default: `None`

9 export

Export options for output files, such as data order, include header in output and hive partitioning.

9.1 fields_to_rename

Rename or remove INFO/tags before exporting.

Type: `dict`

Default: `None`

Examples:

Rename 'CLNSIG' field to 'CLNSIG_renamed' and remove 'SIFT' field:

```
{
   "fields_to_rename": {
     "CLNSIG": "CLNSIG_renamed",
     "SIFT": null
   }
}
```

9.2 order_by

List of columns to sort the result-set in ascending or descending order. Use SQL format, and keywords ASC (ascending) and DESC (descending). If a column is not available, order will not be considered. Order is enable only for compatible format (e.g. TSV, CSV, JSON). Examples: 'ACMG_score DESC', 'PZFlag DESC, PZScore DESC'.

Type: `str`

Default:

Examples:

Order by ACMG score in descending order

```
{
   "order_by": "ACMG_score DESC"
}
```

Order by PZFlag and PZScore in descending order

```
{
   "order_by": "PZFlag DESC, PZScore DESC"
}
```

9.3 include_header

Include header (in VCF format) in output file. Only for compatible formats (tab-delimiter format as TSV or BED).

Default: `False`

9.4 parquet_partitions

Parquet partitioning using hive (available for any format). This option is faster parallel writing, but memory consuming. Use 'None' (string) for NO partition but split parquet files into a folder. Examples: '#CHROM', '#CHROM,REF', 'None'.

Type: `str`

Default: `None`

9.5 force_cast_as_flat

Force cast as flat values (varchar, integer, boolean) for Parquet export. By default, Parquet export preserves all columns type, even as list/nested.

If 'true', values as list will be aggregated within a varchar value, with separator ','.

Type: `bool`

Default: `False`

Examples:

Force cast as flat values for Parquet export:

```
{
   "force_cast_as_flat": true
}
```

10 explode

Explode options for INFO/tags annotations within VCF files.

10.1 explode_infos

Explode VCF INFO/Tag into table columns (e.g. 'variants', 'transcripts').

Default: `False`

10.2 explode_infos_prefix

Explode VCF INFO/Tag with a specific prefix.

Type: `str`

Default:

10.3 explode_infos_fields

Explode VCF INFO/Tag specific fields/tags. Keyword `*` specify all available fields, except those already specified. Pattern (regex) can be used, such as `.*_score` for fields named with '_score' at the end. Examples:

- 'HGVS,SIFT,Clinvar' (list of 3 fields)
- 'HGVS,*,Clinvar' (list of 2 fields with all other fields in the middle)
- 'HGVS,.*_score,Clinvar' (list of 2 fields with all scores in the middle)
- 'HGVS,.*_score,*' (1 field, scores, all other fields at the end)

Type: `str`

Default: `*`

11 samples

Samples parameters to defined a list of samples or use options to check samples. Only for export in VCF format. By default, if no samples are listed, all existing samples are checked if they contain well-formed genotype annotations (based on 'FORMAT' VCF column).

Type: `dict`

Default: `None`

Examples:

Export only a list of samples:

```
{
   "samples": {
      "list": ["sample1", "sample2"]
   }
}
```

Do not check existing samples (all VCF columns after FORMAT column):

```
{
   "samples": {
      "check": false
   }
}
```

Default configuration, with all samples are considered (null) and checked (true):

```
{
   "samples": {
      "list": null,
      "check": true
   }
}
```

11.1 list

List of columns that correspond to samples (with well formed genotype, based on 'FORMAT' VCF column). Only for export in VCF format. Only these samples are exported in VCF format file.

Type: `dict`

Examples:

Export only a list of samples:

```
{
   "list": ["sample1", "sample2"]
}
```

11.2 check

Check if samples (either provided in 'list' parameters, or all existing column after 'FORMAT' column) according to 'FORMAT' VCF column. Only for export in VCF format. By default, samples are checked (beware of format if check is disabled) and removed if they are not well-formed.

Type: `dict`

Examples:

Do not check existing samples:

```
{
   "check": false
}
```

12 databases

HOWARD Parameters Databases JSON describes configuration JSON file for databases download and convert.

13 chunking

Chunking is a technique used to process large files by dividing them into smaller, more manageable pieces (chunks), by specifying chunk size and partitioning scheme. For example, setting this to 1,000,000 will chunk input files with more than 1 million variants into multiple files, each containing up to 1 million variants (only if the input file contains more than 1 million variants). Setting this to '#CHROM' will partition the input file by chromosome. This approach allows each chunk to be processed independently, reducing memory usage, disk usage (duckDB swap), and can help prevent crashes or slowdowns due to memory constraints. Once all chunks are processed, the results are merged to produce the final output.

This method is particularly useful for handling large datasets or files that would otherwise be too resource-intensive to process in one go (such as low memory, or data particularly huge due to extensive annotation). However, splitting, processing, and merging chunks can introduce additional computational overhead, may mislead with some calculations (due to not covering the full region of interest, such as all variants of a gene for compound heterozygote calculation), and can not apply full data features (such as statistics). Moreover, the order of data can change during the chunking-merging process.

Specifically, the 'parquet' chunking mode will split the input file ino multiple parquet files. Another 'duckdb' chunking mode will not split the input file, but will load data into an intermadiate duckDB storage file, reducing memory usage and duckDB swap, and reserving memory to tools and processes.

To enable chunking, include the `chunking` section in the parameters JSON file. This section includes parameters to enable chunking, define chunk size, partitioning scheme, and sorting of the final output.

Examples:

Default configuration for chunking

```
{
   "chunking": {
      "chunking_enable": false,
      "chunking_mode": 'parquet',
      "chunking_size": 1000000,
      "chunking_partitions": "None",
      "chunking_sort": false
   }
}
```

Example of a configuration for chunking big files by chromosome with 100 million variants per chunk

```
{
   "chunking": {
      "chunking_enable": true,
      "chunking_mode": 'parquet',
      "chunking_size": 100000000,
      "chunking_partitions": "#CHROM"
   }
}
```

Example of a configuration for chunking big files with a duckDB intermadiate storage, with chunk_size global parameter as 100 million variants

```
{
   "chunking": {
      "chunking_enable": true,
      "chunking_mode": 'duckdb',
      "chunking_size": 100000000
   }
}
```

13.1 chunking__enable

Chunking process aims to improve processing efficiency, either by:

- splitting input file into smaller parts.
- using duckDB database as a intermediate file storage.

Default: `False`

Examples:

Enable chunking

```
{
   "chunking_enable": true
}
```

Disable chunking

```
{
   "chunking_enable": false
}
```

13.2 chunking__size

Chunking size is the number of variants to process at a time. With 'parquet' chunking mode, input file will be split into multiple smaller Parquet files with a maximum of 'chunking_size' variants per file. With 'duckdb' chunking mode, the global chunk_size parameter will replaceded by 'chunking_size'.

Type: `int`

Default: `1000000`

Examples:

Set chunking size to 1 million variants

```
{
   "chunking_size": 1000000
}
```

Set chunking size to 10 million variants

```
{
   "chunking_size": 10000000
}
```

13.3 chunking__partitions

Partitioning scheme to use with 'parquet' chunking mode, defining how the input file is partitioned during chunking.

Type: `str`

Default: `None`

Examples:

No partitioning

```
{
   "chunking_partitions": "None"
}
```

Partition by chromosome

```
{
   "chunking_partitions": "#CHROM"
}
```

13.4 chunking_sort

Sort final output after chunking is applied. Prevents issues with unordered variants after chunked processing.

Default: `False`

Examples:

Enable sorting

```
{
   "chunking_sort": true
}
```

Disable sorting

```
{
   "chunking_sort": false
}
```

14 threads

Specify the number of threads to use for processing HOWARD. It determines the level of parallelism, either on python scripts, duckdb engine and external tools. It and can help speed up the process/tool. Use -1 to use all available CPU/cores. Either non valid value is 1 CPU/core.

Type: `int`

Default: `-1`

Examples:

Automatically detect all available CPU/cores

```
{
   "threads": -1
}
```

Define 8 CPU/cores

```
{
   "threads": 8
}
```

15 fast

Fast annotation mode (Experimental). Speedup export and reduce memory. Only Parquet annotation will be processed.

Default: `False`